

Lecture - 31

Introduction to 2D Array

2-dimensional Array

```
int arr[4][3];
```

rows column

	columns			
	0	1	2	
0	00	01	02	} rows
1	10	11	12	
2	20	21	22	
3	30	31	32	

chessboard, Tic Tac Toe

*in memory

		00	01	02	10	11
12	20	21	22	30	31	32

Memory

Row Major

stored at contiguous memory location

① Row Major

	0	1	2
0	00	01	02
1	10	11	12
2	20	21	22
3	30	31	32

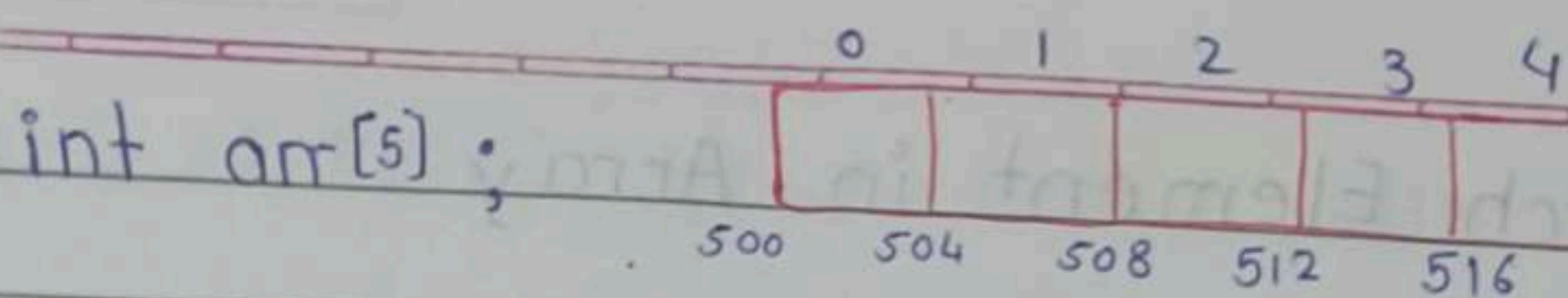
0	1	2	3	4	5	6	7	8	9	10	11
00	01	02	10	11	12	20	21	22	30	31	32

memory

Index = row-index $\times$ col + col-index

• row-index = $\frac{\text{Index}}{\text{col}}$

• col-index = $\text{Index} \% \text{col}$



$$\begin{aligned} \text{arr}[3] &= \text{base-address} + \text{index} \times \text{size of element} \\ &= 500 + 3 \times 4 \\ &= 512 \end{aligned}$$

$$\text{arr}[2][1] = \text{base-address} + \text{Index} \times \text{size of element}$$

500 *

loop points

$$\text{arr}[i][j] = \text{base-address} + \underbrace{(i \times \text{col} + j)}_{\text{index}} \times \text{size of element}$$

Working on 2D Array

// Create a 2D Array

int arr[4][3];

// Initialise

int arr[4][3] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12};

// Update or User Input

arr[3][0] = 15

cin >> arr[1][2]

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9
3	10	11	12

// Print

for (int i = 0; i < 4; i++) {

for (int j = 0; j < 3; j++) {

cout << arr[i][j];

}

Problem - 1 : Search Element in Array

```
int arr[4][3] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12}
```

```
int x = 7
```

```
for (int i = 0 ; i < 4 ; i++) {
```

```
    for (int j = 0 ; j < 3 ; j++) {
```

```
        if (arr[i][j] == x)
```

```
            return 1;
```

```
    }
```

```
}
```

11 2D Array Function

```
void print (int arr[][4], int row, int col) {
```

```
    for (int i = 0 ; i < row ; i++) {
```

```
        for (int j = 0 ; j < col ; j++) {
```

```
            cout << arr[i][j];
```

```
        }
```

```
    }
```

11 function Call : `print (arr, 4, 3);`

Problem - 2 : Add 2 matrix

	2	3	4	5		3	4	1	2		5	7	5	7	
0 (row x	0	1	2	3	+	2	2	2	0	=	2	3	4	3	
column)	1	7	6	4		1	3	5	4		2	5	11	8	x
	arr1[3][4]				+	arr2[3][4]				=	ans[3][4];				

```
ans[0][0] = arr1[0][0] + arr2[0][0];
```

Problem - 3 : Print Row Index with Maximum Sum

	0	1	2	3	
0	3	4	7	18	$\Rightarrow 32$ <u>0</u>
1	2	8	3	9	$\Rightarrow 22$
2	5	4	2	2	$\Rightarrow 13$
3	7	3	0	8	$\Rightarrow 18$
4	2	8	9	1	$\Rightarrow 30$

```

int sum = INT-MIN, index = 0;
for (int i = 0; i < row; i++) {
    int total = 0;
    for (int j = 0; j < col; j++) {
        total += arr[i][j];
    }
    if (sum < total) {
        sum = total;
        index = i;
    }
}

cout << index;

```

Problem - 4 : Print Sum of Diagonal Element

5	8	3	9
6	2	8	4
5	3	2	2
2	1	1	9

row = column

$$5 + 2 + 2 + 9 = 18 \text{ first}$$

$$9 + 8 + 3 + 2 = 22 \text{ second}$$

```

int second = 0, i = 0, j = col - 1;
while (j >= 0) {
    second += arr[i][j];
    i++, j--;
}

```

T.C : $O(\text{row})$

S.C : $O(1)$

Problem -5 : Reverse each row of matrix

2	3	4	5	⇒	5	4	3	2
1	2	6	8	⇒	8	6	2	1
4	9	3	2	⇒	2	3	9	4

2	3	4	5
---	---	---	---

↑ start ↑ end

```
for (int i=0 ; i<rows ; i++) {
    start = 0, end = col-1;
    while (start<=end) {
        swap(arr[i][start],
              arr[i][end]);
        start++;
        end--;
    }
}
```

Lecture - 32

Column Major Order

	0	1	2		0	1	2	3	4	5	6	7	8	9	10	11	12
0	1	2	3														
1	4	5	6		1	2	3	4	5	6	7	8	9	10	11	12	
2	7	8	9														
3	10	11	12														

arr[2][2]

arr[2][2]

Find formula of index

arr[4][3]

Vector in 2D

// Create

	0	1	2	3
0	2	2	2	2
1	2	2	2	2
2	2	2	2	2

vector<vector<int>> matrix;
vector<vector<int>> matrix (rows,
vector<int> col, i)

matrix

Find no. of rows
↑
2 cols

// update

matrix[1][2] = 5

// User-Input

cin >> matrix[2][2];

row = matrix.size();

col = matrix[0].size();

Problem 1 : Wave Form

	0	1	2	3
0	3	6	4	2
1	7	8	11	5
2	9	3	2	1
3	17	8	5	9

3 7 9 17 8 3 8 6 4 11 2 5 9 15 2

col = even (up → down)

col = odd (down → up)

S.C = O(1)

T.C = O(row * col)

```
for (int j=0 ; j<col ; j++) {
    if (j%2 == 0) {
        for (int i=0 ; i<row ; i++)
            cout << arr[i][j];
    } else {
        for (int i=row-1 ; i>=0 ; i--)
            cout << arr[i][j];
    }
}
```

Problem 2 : Spiral Form

	0	1	2	3	4	5	↓ Right
Top → 0	1	2	3	4	5	6	
1	7	8	9	10	11	12	
2	13	14	15	16	17	18	
3	19	20	21	22	23	24	
4	25	26	27	28	29	30	
5	31	32	33	34	35	36	← Bottom
	↑ Left						

Top row	Right	bottom	left
= 0	column	row	column
	= 5	= 5	= 0

```

while (top <= bottom && left <= right) {
    for (int j = left; j <= right; j++)
        cout << matrix[top][j];
    top++;
    for (int i = top; i <= bottom; i++)
        cout << matrix[i][right];
    right--;
    for (int i = right; i >= left; i--)
        cout << matrix[bottom][i];
    bottom--;
    for (int i = bottom; i >= top; i--)
        cout << matrix[i][left];
    left++;
}

```

Problem - 3 : Transpose Matrix

	0	1	2	3					
0	1	2	3	4		1	5	9	13
1	5	6	7	8	⇒	2	6	10	14
2	9	10	11	12		3	7	11	15
3	13	14	15	16		4	8	12	16

Matrix

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        arr[i][j] = matrix[i][j];
    }
}
```

T.C : $O(N^2)$
S.C : $O(N^2)$

```
for (int i = 0; i < row - 1; i++) {
    for (int j = i + 1; j < col; j++) {
        swap(matrix[i][j], matrix[j][i]);
    }
}
```

T.C : $O(N^2)$
S.C : $O(1)$

Homework Questions

- 1) Use lower triangle for transpose
- 2) Reverse matrix ($n \times m$)

1	2	3	⇒	7	8	9
4	5	6		4	5	6
7	8	9		1	2	3
- 3) Spiral matrix
- 4) Print diagonally
- 5) Print matrix in snake pattern

Lecture : 33

ROTATE MATRIX

Problem 1 : Rotate Matrix by 90° clockwise / Rotate Image

	0	1	2	3					
0	1	2	3	4	90°	13	9	5	1
1	5	6	7	8		14	10	6	2
2	9	10	11	12		15	11	7	3
3	13	14	15	16		16	12	8	4

ans[n][n];

ans[n][n];

Matrix : $n \times n$

$i \rightarrow j$ $x = n-1-i$
[0][0] $\rightarrow$ [0][3]

[0][1] $\rightarrow$ [1][3]

[0][2] $\rightarrow$ [2][3]

ans[j][n-1-i] = matrix[i][j];

```
for (int i=0 ; i<n ; i++) {
    for (int j=0 ; j<n ; j++) {
        ans[j][n-1-i] = matrix[i][j];
    }
}
```

T.C : $O(n^2)$

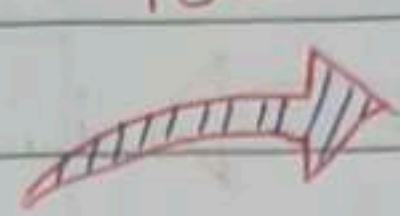
S.C : $O(n)$

{ (col $\rightarrow$ reverse) $\Rightarrow$ Row }

```
for (int i=0 ; i<n-1 ; i++) {
    for (int j=i+1 ; j<n ; j++) {
        swap(matrix[i][j], matrix[j][i]);
    }
}
```

```
for (int i=0 ; i<n ; i++) {
    start = 0, end = n-1;
    while (start < end) {
        swap(matrix[i][start], matrix[i][end]);
        start++, end--;
    }
}
```

Problem - 2 : Rotate Matrix 180°

	0	1	2	3		0	1	2	3	
0	1	2	3	4	180°	0	16	15	14	$90^\circ + 90^\circ$
1	5	6	7	8		1	12	11	10	1
2	9	10	11	12		2	8	7	6	Transpose Trans
3	13	14	15	16		3	4	3	2	+

matrix

```

for (int j = 0 ; j < n ; j++) {
    start = 0 , end = n-1 ;
    while (start < end) {
        swap(matrix[start][j] , matrix[end][j]) ;
        start ++ , end -- ;
    }
}

```

// pehle column ke wise reverse krlo

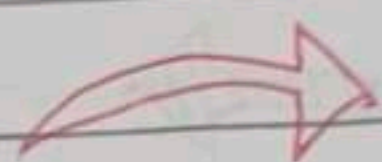
```

for (int i = 0 ; i < n ; i++) {
    start = 0 , end = n-1 ;
    while (start < end) {
        swap(matrix[i][start] , matrix[i][end]) ;
        start ++ , end -- ;
    }
}

```

Problem - 3 : Rotate Matrix 90° anticlockwise

1	2	3	4		4	8	12	16
5	6	7	8		3	7	11	15
9	10	11	12		2	6	10	14
13	14	15	16		1	5	9	13



H.W :

$$360^\circ - 90^\circ = 270^\circ$$

(i) Transpose

1	5	9	13
2	6	10	14
3	7	11	15
4	8	12	16

column-wise

90°

reverse

anticlockwise

$\Rightarrow$ ek code hoga transpose ka aur krne ka column ka

Problem - 4 : Rotate Matrix by k Time

```

k = k % 4;
while (k) {
    Rotate  $90^\circ$  (matrix); // Function
    k--;
    to rotate  $90^\circ$ 
}

```

- 1 time - 90°
- 2 time - 180°
- 3 time - 270°
- 4 time - 360°
- 5 time - 90°
- 6 time - 180°

2

Lecture-34

	0	1	2	3	4
0	2	6	10	14	18
1	20	24	27	29	38
2	47	52	78	93	102
3	108	111	200	218	320

$N = 4, M = 5, X = 52$

```
for (int i=0 ; i<N ; i++) {
    for (int j=0 ; j<M ; j++) {
        if (Matrix[i][j] == X)
            return 1;
    }
}
```

```
* for (int i=0 ; i<N ; i++) {
    {
        return 0;
    }
}
```

```
if (Matrix[i][0] <= X && X <= Matrix[i][M-1]) {
```

```
    start = 0, end = M-1;
```

```
    while (start <= end) {
```

```
        mid = (start + end) / 2;
```

```
        if (Matrix[i][mid] == X)
```

```
            return 1;
```

```
        else if (Matrix[i][mid] < X)
```

```
            start = mid + 1;
```

```
        else
```

```
            end = mid - 1;
```

```
    } }
}
```

```
return 0;
```

$\log M$

$T.C = O(N + \log M)$

$S.C = O(1)$

0	1	2	3	4	5	6	7
2	6	10	14	18	20	24	27

1D Array me assume krlo

```
* start = 0, end = N*M-1;
```

```
while (start <= end {
```

```
    mid = (start + end) / 2;
```

```
    row-index = mid / M;
```

```
    col-index = mid % M;
```

```
    if (Matrix[row-index][col-index] == X)
```

```
        return 1;
```

```
    else if (Matrix[row-index][col-index] < X) start = mid + 1;
```

```
    else end = mid - 1;
```

$T.C: O(\log(NM))$

$S.C: O(1)$

Problem - 2 : Search in Sorted Row & Col Matrix

	0	1	2	3	4
0	4	8	15	25	60
1	18	22	26	42	80
2	36	40	45	68	104
3	48	50	72	83	130
4	76	99	114	128	170

Matrix

$N=5, M=5, X=50$,

$j=0, j=M-1$;

while ($i < N$ & $j \geq 0$) {

if ($Matrix[i][j] == X$)

return 1;

else if ($Matrix[i][j] < X$)

$i++$;

else

$j--$;

return 0;

T.C : $O(N+M)$

$\sim O(1)$

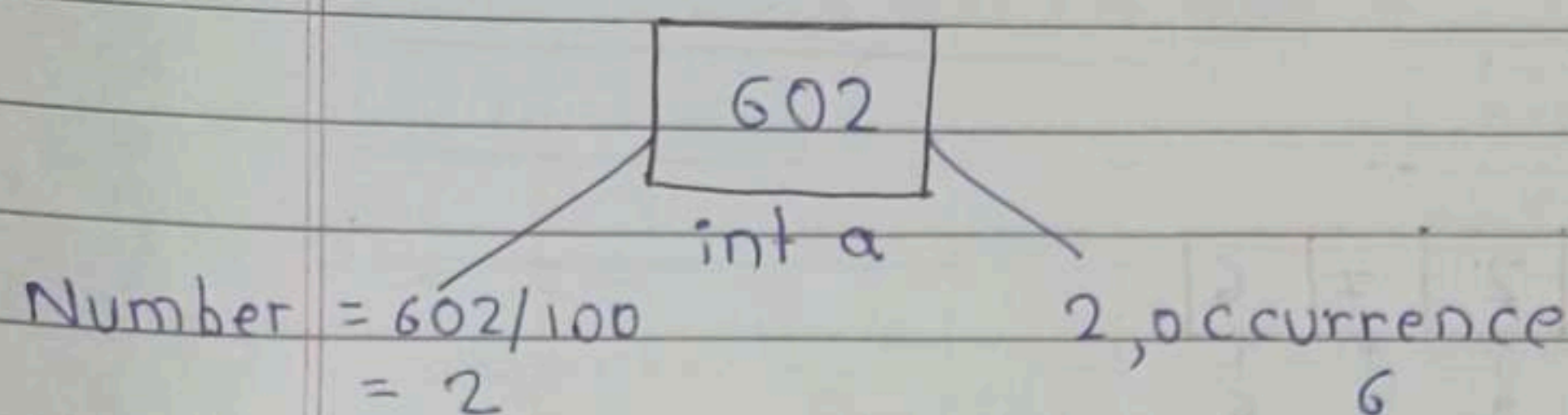
S.C : $O(1)$

Homework

- 1) Count zeroes in a sorted matrix
- 2) Row with max 1s
- 3) Binary Search : (solve it in $\log(n) + \log(m)$ time, where BS in a 2D Array which is sorted in decreasing order. N is the no. of rows and M is the no. of columns)

Lecture-35

How to store 2 number in 1 position



$$N = 1 + 099$$

Store no's & its occurrence

2 2 2 2 2

Occurrence =

$$= 602/100$$

$$= 6$$

	0	1	2	3
0	2	4	8	10
1	12	14	16	18
2	20	22	24	26

0	1	2	3	4	5	6	7	8	9	10	11
2	4	8	10	12	14	16	18	20	22	24	26

7

row cols

$$7/4 = 1$$

$$7 \% 4 = 3$$

Index = Row-index $\times$ col + col-index

Problem 1 - Find missing and Repeating

arr

0	1	2	3	4	5	6
4	3	2	1	2	7	6

$N = 7$ 1 to N 1 to 7

Brute Approach: Repeating = 2 } $O(n^2)$

Missing = 5 } $O(n^2)$

2nd Approach: 1 2 2 3 4 6 7 (sort) $N \log n$

Missing = 5 } $N \log N$

Repeat = 2 }

IIIrd Approach -

0	1	2	3	4	5	6
4	3	2	1	2	7	6

count

0	1	2	3	4	5	6	7
0	1	2	1	1	0	1	1

Repeat = 2

Missing = 5

arr =	4	3	2	1	2	7	6
	↑	↑	↑	↑	↑	↑	↑
	0	1	2	3	4	5	6

count =	1	2	1	1	0	1	1
---------	---	---	---	---	---	---	---

$$4-1=3$$

```
vector<int> count(N, 0);
for (int i=0 ; i<N; i++) {
    count[arr[i]-1]++;
}
```

II Missing Element

```
for (int i=0 ; i<N; i++) {
    if (count[i] == 0) {
        cout << i+1;
        break;
    }
}
```

$O(N)$

II Repeat Element

```
for (int i=0 ; i<N; i++) {
    if (count[i] == 2) {
        cout << i+1;
        break;
    }
}
```

$O(N)$

T.C : $O(N)$

S.C : $O(N)$

arr =	4	3	2	1	2	7	6
-------	---	---	---	---	---	---	---

arr =	3	2	1	0	1	6	5
-------	---	---	---	---	---	---	---

$$2+7 \times 2 = 16 \% 7 \Rightarrow 2$$

$$16/7 = 2$$

$$6+7 \times 1 = 13 / 7 = 1$$

$$13 \% 7 = 6$$

0	1	2	3	4	5	6
10	16	8	7	1	13	12

$$3 \% 7 = 3 \rightarrow +7$$

original
arr[i] % 7

occurrence
arr[i] / 7 ;

```
for (int i = 0; i < N; i++) — N
    arr[i] -- ;
```

```
for (int i = 0; i < N; i++) — N
    arr[arr[i] % N] += N; — N + N + N + N = 4N
```

// Missing No for (int i = 0; i < N; i++) { — N T.C = O(N)
 if (arr[i] / N == 0) S.C = O(1)
 cout << i + 1;
 break;
 }

// Repeating No for (int i = 0; i < N; i++) { — N
 if (arr[i] % N == 2)
 cout << i + 1;
 break;
 }

Problem 2 : Find occurrence of Number

0	1	2	3	4	5	6	7
3	2	5	3	1	2	3	7

N = 8

1 to N

1 - 1

2 - 2

3 - 3

4 - 0

5 - 1

6 - 0

7 - 1

8 - 0

frequency

arr

0	1	2	3	4	5	6	7
2	1	4	2	0	1	2	6

count

0	1	2	3	4	5	6	7
1	2	3	0	1	0	1	0

4 extra

arr created

Problem 3 : Majority Element

0	1	2	3	4	5	6	7	8	9	10	
3	3	2	3	1	3	2	2	1	3	3	N = 11

→ $N/2$

→ 5

1 1 2 2 3 3 3 3 3 3 3 ($n \log n$)

candidate = 3

count = 1 2 1 2 1 2 1 2 1 2 1

2	2	2	2	3	3	3	3	3
---	---	---	---	---	---	---	---	---

2 3

2 3

2 3

2 3

3 ✓

$O(N)$

Moore voting

algorithm

[Verify → $N/2$]

```
int candidate, count = 0;
```

```
for (int i = 0; i < N; i++) {
```

```
    if (count == 0) {
```

```
        count = 1;
```

```
        candidate = arr[i];
```

```
    } else {
```

```
        if (candidate == arr[i])
```

```
            count ++;
```

```
        else
```

```
            count --;
```

```
    }
```

```
for (int i = 0; i < N; i++) {
```

```
    if (arr[i] == candidate) count ++;
```

```
    if (count > N/2) return candidate;
```

```
    else return -1;
```

```
}
```